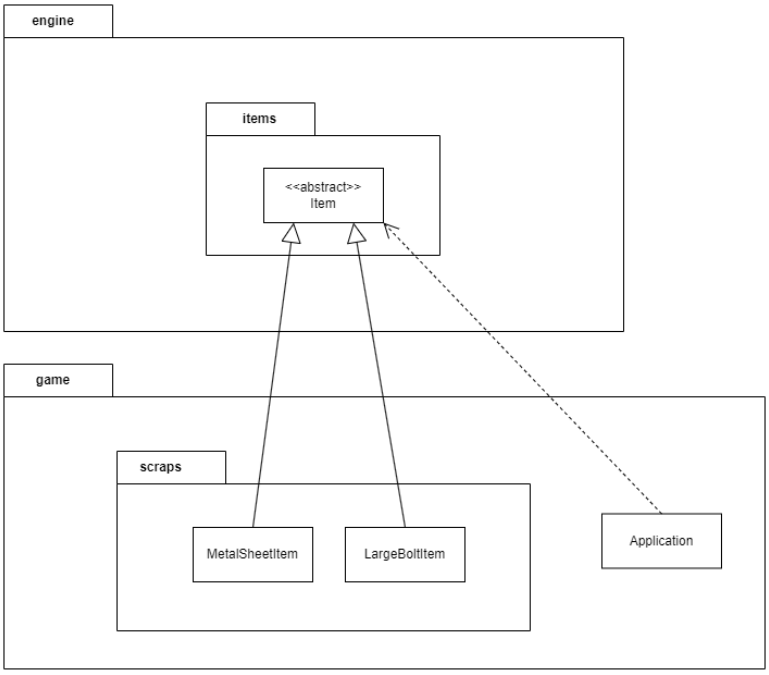
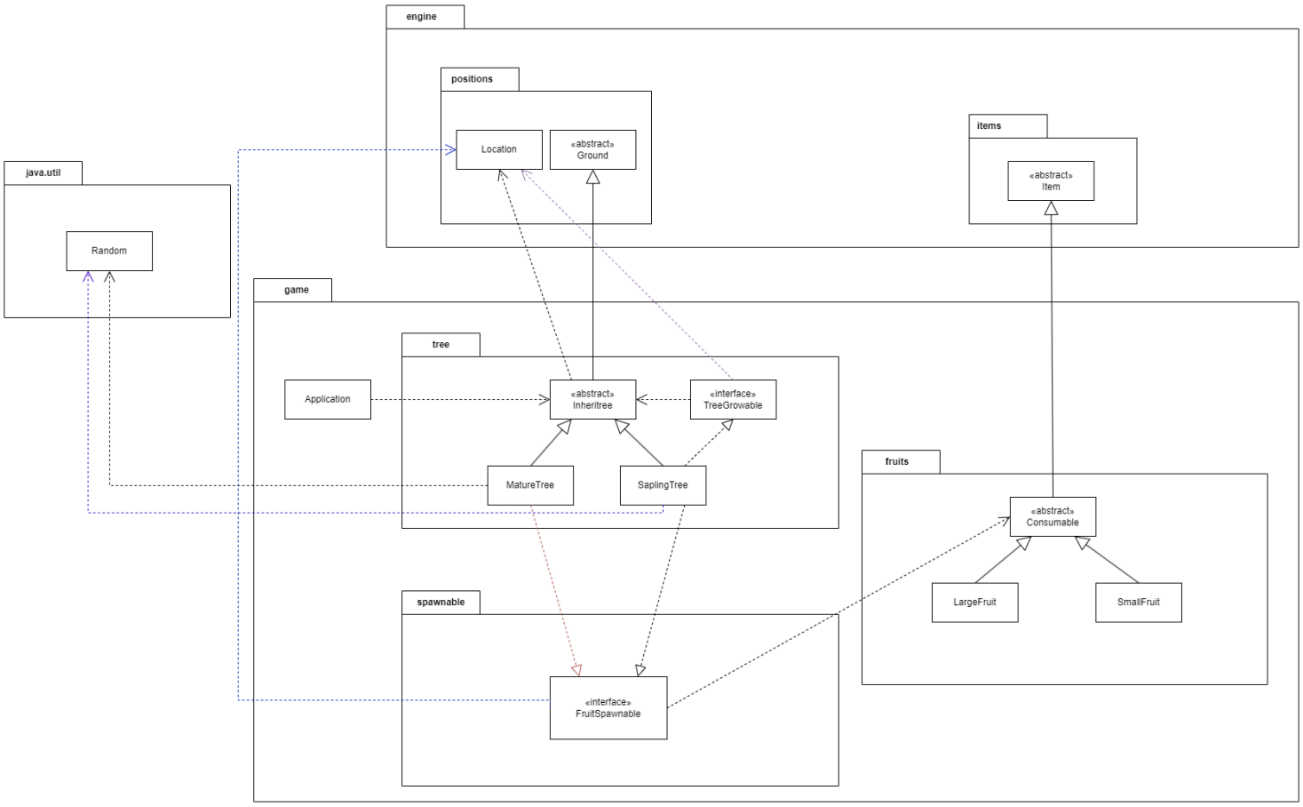
FIT2099 Assignment 1 Design Documentation

Design Diagrams

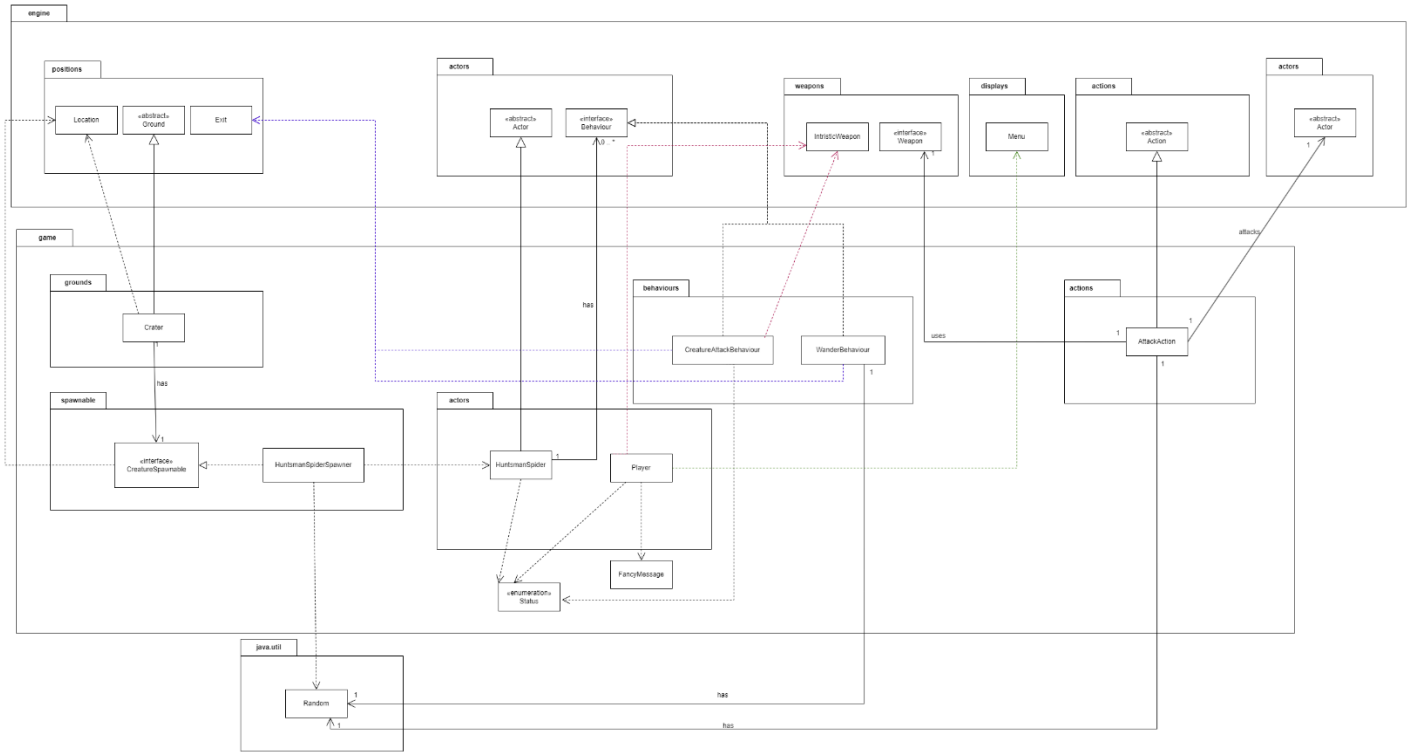
REQ 1 UML Class Diagram:



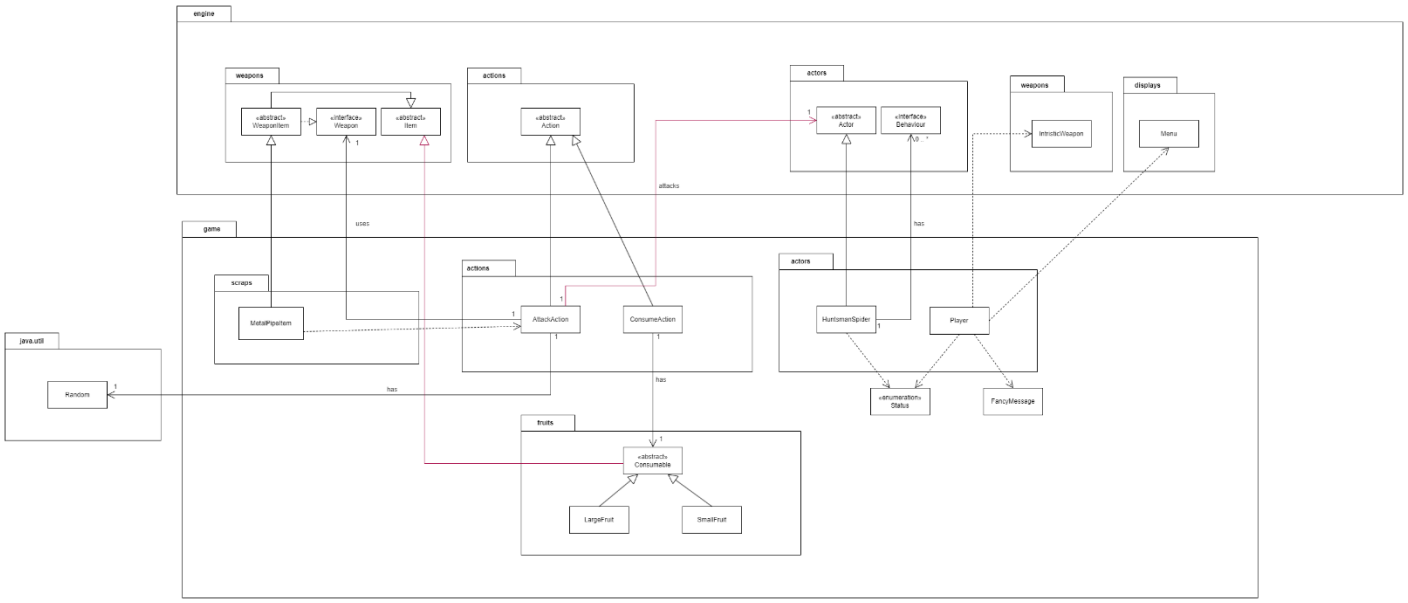
REQ 2 UML Class Diagram:



REQ 3 UML Class Diagram:



REQ 4 UML Class Diagram:



Design Rationale

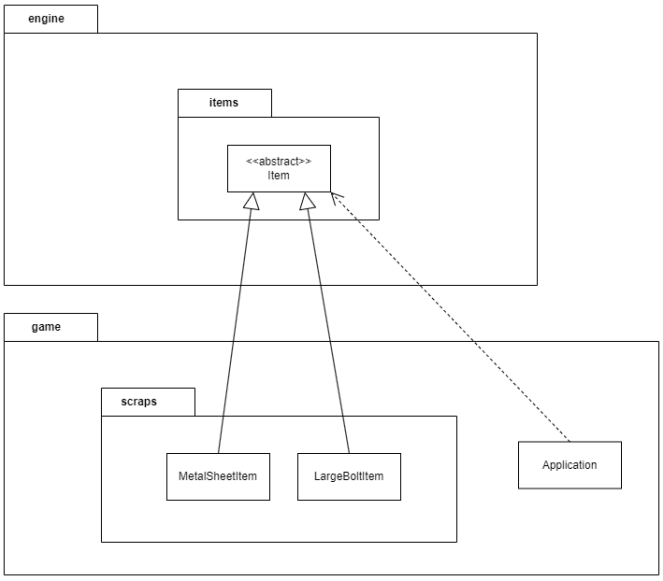
<u>REQ 1</u>

Design 1: Create concrete classes that represent the items Large Bolt and Metal Sheet without extending any super classes.

Pros	Cons
Simpler & flexible, able to only implement the functionality that the respective Items need without the need of inheriting functionality.	Much repeated code, both classes represent Items and have the same traits, only the character to represent them is different.
	Not fully utilizing the given engine, abstract class Item in engine already gives the base methods and attributes for an Item in the game.

Design 2: Create concrete classes that represent the items Large Bolt and Metal Sheet by extending the given abstract class Item in the engine.

Pros	Cons
Promotes code reuse, as it utilizes the given abstract class Item in engine, which already provides the base methods and attributes that both Items need.	Extending an abstract class introduces tight coupling between the concrete class and the abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.
Future extension of common methods only needs to be added to the abstract super class, child classes will inherit the methods with no need to modify child classes.	
Easily scalable, if more items are to be added to the game later, we can easily add new classes by extending the abstract class Item.	



The diagram above shows the final design of requirement 1, which utilizes Design 2 stated above.

Both new items created extend the abstract Item class. Since these items share many common attributes and methods, it is logical to abstract these identities to avoid repetitions in code (DRY). The new items created solely use the constructor of the super class to initialize their object, making them have the same attributes and methods of the super class with only different initialized values, enabling them to replace the usage of their subclass in code (Liskov Substitution Principle).

The main game Application has a dependency on abstract class Item, removing extra dependencies that would result if items were designed as concrete classes which do not extend Item. Application is a high-level module in the game as it executes the main game loop, whilst items are considered low-level modules which implement game item logic. Removing dependencies of Application on these concrete item classes reduce the risk of Application being affected by changes in low-level modules (Dependency Inversion Principle).

REQ 2

A. Implementation of Inheritree & plants that can spawn fruit

Design 1: Create an abstract class Inheritree by extending the given abstract Ground class, which would be the base class of a plant. Then, create another abstract class FruitTree that extends Inheritree for plants that can spawn fruits at nearby exits for each tick, with concrete instances of different fruit trees extending this class.

Pros	Cons
Easy implementation and simple logic, clear representation of relationship between classes.	Multiple layers of abstraction would increase code complexity and result in higher chances of parent class method logic being lost in its deeply nested child classes.
	Extending abstract classes creates a tight coupling between subclasses and their superclass, changes to the superclass may affect all subclasses, causing maintenance issues in multi-level inheritance as there are more classes to maintain.
	FruitTree class may have multiple responsibilities, such as managing tree growth and spawning fruits.

Design 2: Create an abstract class Inheritree by extending the given abstract Ground class, which would be the base class of a plant. Concrete classes of different plants would extend this abstract class. For plants that can spawn fruits, their classes implement another interface FruitSpawnable that provides base methods for the classes to implement to spawn fruits at nearby exits for each tick.

Pros	Cons
Code is more modular, the abstract class defines common behaviour for all plants, while the interface provides a contract for plants that can spawn fruits.	May be redundant to use an interface for plants that could grow if all plants created implement this interface.
More flexible approach, concrete plant classes can choose to implement the FruitSpawnable interface if they can spawn fruits, but it's not a must, promoting a more adaptable and extensible system. Extensions for plants to spawn multiple fruits can also be	

implemented by only modifying the logic of fruit spawning in the individual plant classes.	
Enhances code readability and maintainability as interfaces define a clear contract for classes that implement them, making it easy to understand what behaviour is expected from classes that implement the interface.	

B. Implementation of Consumable (Fruit)

Design 1: Using an interface Consumable, then create concrete classes of different fruits by extending Item abstract class in the engine and using the interface.

Pros	Cons
Easier to add different functionalities to the fruits besides being Consumable, more extensible in future developments.	Unable to group similar attributes using an interface, each concrete class would have to initialise their own set of attributes.
Enforce contract for fruits to have the same set of methods, but able to have different effects for different fruits, since interfaces do not define the method.	More repeated code, methods enforced that have the same logic would still need to be implemented in all concrete classes that use the interface.

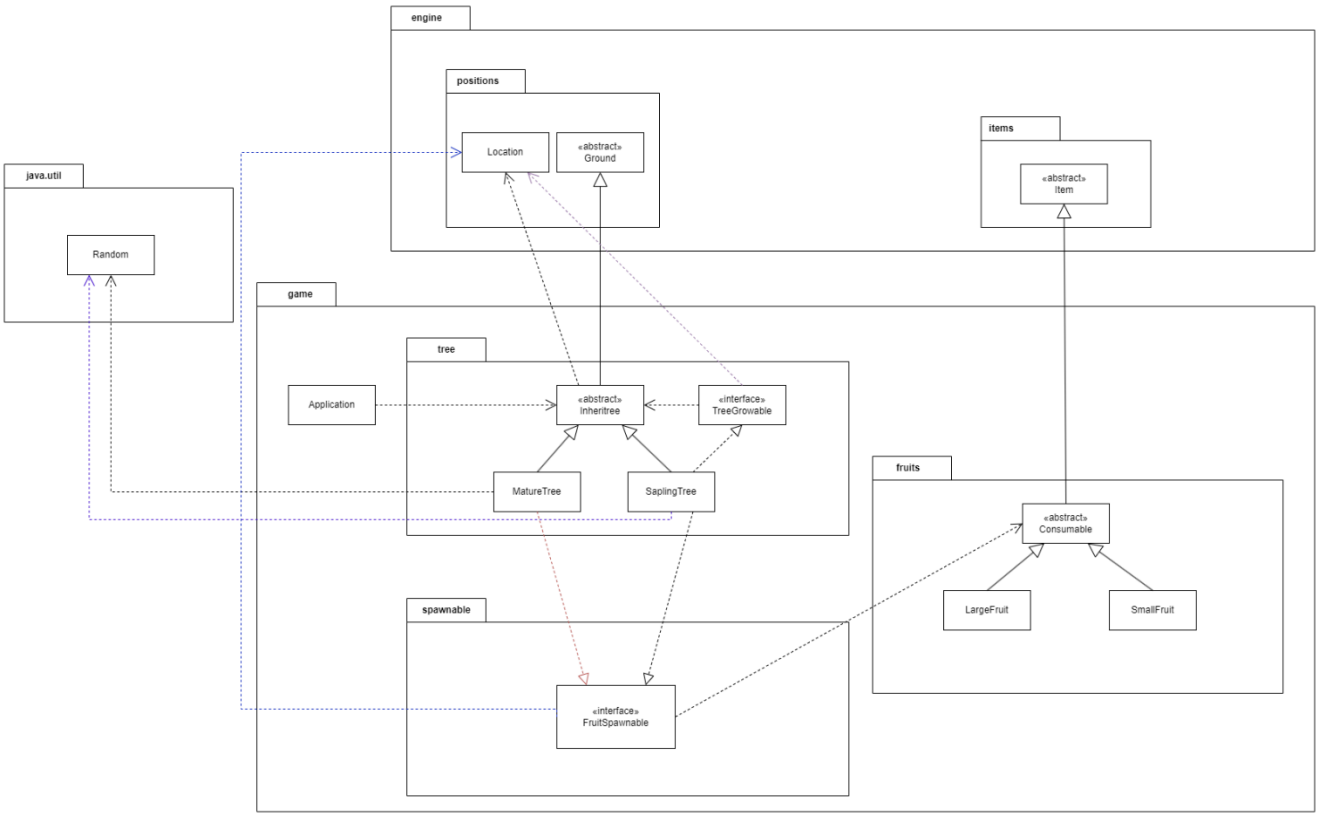
Design 2: Using an abstract class Consumable which extends Item in the engine, then then create concrete classes of different fruits by extending the abstract class.

Pros	Cons
Shared attributes and methods of (Consumable) fruits can be defined in the abstract class, concrete classes only need to inherit the attributes and methods, less repeated code for fruits that have highly similar traits.	Limited inheritance to only 1 abstract class per concrete class, limits the development of functionality of each individual concrete class.
Easy to expand common attributes and methods via the abstract class, without modifying concrete classes.	

C. Growing the next stage of the plant

Growing a tree to the next stage is done by implementing an interface TreeGrowable, which contracts methods to obtain the next stage and replace the plant at the location with a plant of the next stage. The tick() method of a tree is overridden to replace the original location with another type of tree using the interface methods after a certain amount of ticks.

Pros	Cons
Plants that can grow can implement this interface to ensure a contracted set of methods are available in the concrete class to grow the plant to the next stage.	The growth behaviour being triggered by the tick() method of the plant could introduces a dependency between the growth mechanism and the tree's internal timing mechanism, which would be harder to maintain.
More flexible, different plants can have different growth patterns and requirements while also maintaining the contract of the interface. Plants already extend the abstract class Inheritree, using interfaces to contract more extensions to plant classes would not contradict the inheritance of the existing abstract Inheritree class.	



The diagram above shows the final design of requirement 2, which utilizes each Design 2 of parts A, B and design of part C stated above to implement Inheritree plant, Consumable fruits, growing the plants and spawning of these fruits on the exits of the plant.

The new abstract Inheritree class for plants extends the abstract Ground class, while the new abstract Consumable class for consumable fruit items extends the abstract Item class. Since we want the Inheritree to also act as a type of Ground for actors to step on and the fruits to act as items that the actor can perform actions with, it is logical to abstract the identities to avoid repetitions in code (DRY).

Using different interfaces to define different traits (i.e. can grow) and abilities (i.e. ability to spawn fruits) of the concrete plant instances allows us to separate different responsibilities to different interfaces, resulting in better maintainability of the system (Single Responsibility Principle). This approach also allows more extensions to the plants, by adding new interfaces that define new traits since the current interfaces are already separated by their own purposes (Interface Segragation Principle).

FruitSpawnable interface has a dependency on Consumable abstract class. This implementation allows us to remove multiple dependencies of the Consumable class on multiple concrete plant classes that can spawn Consumable fruits. When introducing more new Consumable fruit items, we do not need to fix the concrete classes to be able to use the new fruit items as the concrete plant classes already implement the FruitSpawnable interface (Open-Close Principle).

REQ 3

A. Implementation of Crater

New Crater class that extends abstract Ground class of engine.

Pros	Cons
Inheriting from abstract Ground class allows reusability of existing code and less repeated code and clearer implementation of Crater class.	Tighter coupling is introduced by using inheritance, increasing code complexity and harder to implement further extensions.

B. Changes to original implementation of HuntsmanSpider

1. Map used to store behaviours changed from HashMap to TreeMap:

   The design opted to use a TreeMap to store behaviours as the weight which determines the order of execution of behaviours is important, while a HashMap does not guarantee the order of given elements returned.

	HashMap	TreeMap
Pros	Efficient, constant-time performance of basic operations.	Ordered, TreeMap maintains the elements in sorted order based on a natural ordering or a custom comparator.
Cons	Iterating over the elements of a HashMap is unordered.	Slower performance and uses more memory to store.

2. Add status HOSTILE_TO_PLAYER to not allow the HuntsmanSpider on the spaceship Floor.

   Easy to implement for all types of creature in future creation, allows the HuntsmanSpider to not attack creatures that also attack the player, but might need to change the implementation of attack logic if creatures added are hostile to any other actor including the player.

C. Changes to original implementation of Floor

Design 1: Use `instanceof` and downcasting to check the type of actor that wants to enter, then return true/false based on the type of actor identified.

Pros	Cons
Easy and clear implementation.	Hard to extend more types of actors in the game, code would have to be modified to add checks for every newly added instance, reduces code clarity and maintainability.

	Overuse of `instanceof` and downcasting can be considered a code smell, as it often indicates a violation of the Open-Closed Principle.

Design 2: Override canActorEnter() method to not allow actors who have the status HOSTILE_TO_PLAYER to enter the spaceship floor.

Pros	Cons
Can check which actor instances can enter the Floor by checking for a specific status that the actor has, easier to extend by adding a status to actors that we do not want to allow into the spaceship.	Additional status logic implementation in actor classes is needed to allow this approach to work.
Floor class is responsible for defining which actors are allowed to enter, any future modifications to entering conditions can be implemented by just changing the status.	canActorEnter() method is an inherited method from Ground abstract class, overriding this method adds complexity in design as we need to make sure the override does not change the core behaviour of the original super class method.

D. Creature attack if player enters creature's surroundings & show FancyMessage when player is dead

Creating a new CreatureAttackBehaviour class that implements Behaviour interface, which is a behaviour of the creature to attack the player if the player enters creature's surroundings.

Pros	Cons
Implementing Behaviour interface provides a clear contract of methods that the CreatureAttackBehaviour should implement.	May introduce redundancy in simple attack logic implementations, as actor's playTurn() method can also be used for checking and returning attack actions.
Modularizes code, can be used by any other creature that is created in future work to initiate attacks when a target is in its surroundings.	

Modify Player class isUnconcious() method to add a FancyMessage string when the player's hit points is 0, but downfalls of this is that the FancyMessage will show every

time the player's hit points is 0, regardless of what attack resulted in the death of the player.

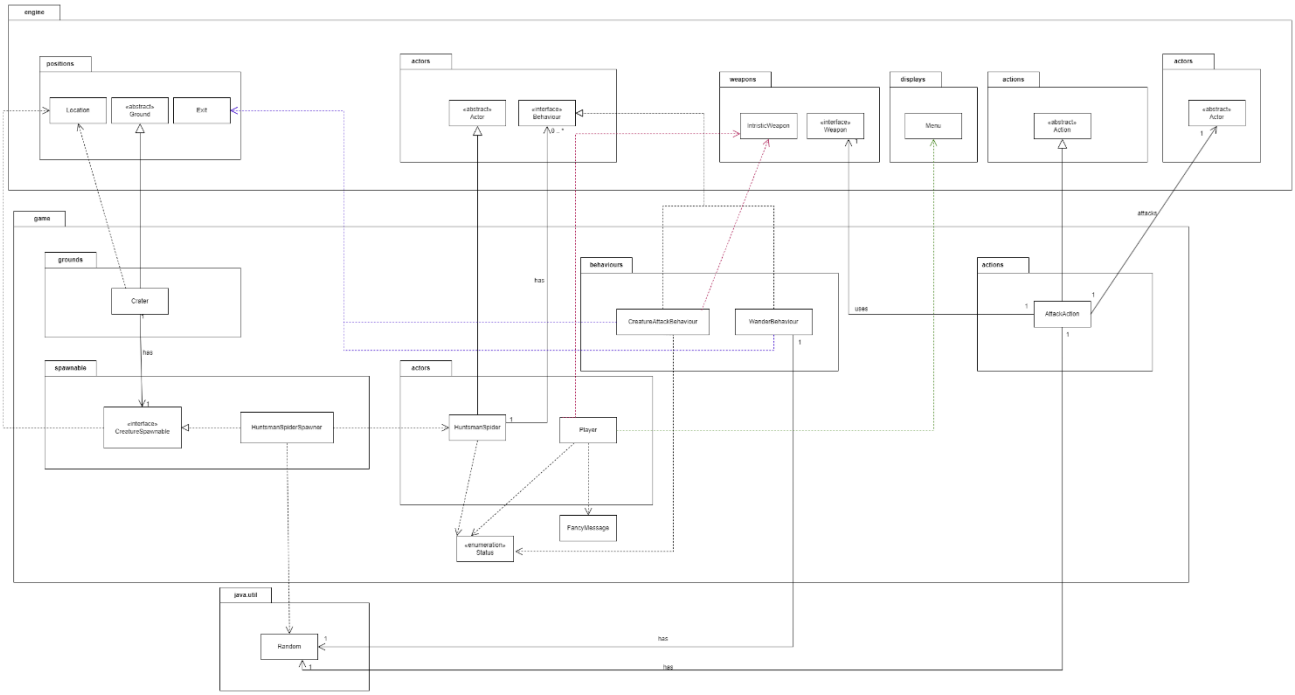
E. Spawning creatures randomly at each tick of game

Design 1: Add spawnCreature() method in class Crater

Pros	Cons
Easy implementation.	Crater would have multiple responsibilities tied, checking if the actor can enter, checking for spawning a creature at an exit using the given spawn chance and spawning the creature.
	For every entity that can spawn creatures, we need to independently implement spawning logic in all of the entity classes.

Design 2: Create interface CreatureSpawnable to provide methods for entities that can spawn creatures, then utilizing the interface to create a concrete spawner class of a specific creature to be used in class Crater.

Pros	Cons
Interface CreatureSpawnable provides a clear blueprint of methods that need to be implemented to spawn creatures.	Needs multiple spawner classes if entities are extended to spawn multiple creatures in future extensions.
Modularizes code, segregates the responsibility of spawning creatures to another class.	
Dependency on the interface in Crater class reduces dependencies on the concrete spawner class.	



The diagram above shows the final design of requirement 3, which utilizes each Design 2 of parts C, E and design of parts A, B, D stated above to implement Crater class that spawns HuntsmanSpiders, implement HuntsmanSpider class that attacks when player enters creature's surroundings and blocking the HuntsmanSpider from entering the spaceship.

The new Crater class extends the abstract Ground class, HuntsmanSpider class extends abstract Actor class, AttackAction class extends abstract Action class. All these classes inherit their methods and attributes from existing abstract classes provided in the engine. We abstract these identities to avoid repetitions in code (DRY), as these classes share many same attributes and methods with the respective abstract classes they extend.

Crater class has an attribute of type interface CreatureSpawnable, which associates Crater class with the interface. The interface acts as a level of abstraction between Crater class and HuntsmanSpiderSpawner class, removing dependencies between the two concrete classes without changing the flow of control by diverting the dependency of both concrete classes to depend on the CreatureSpawnable interface (Dependency Inversion Principle). It also increases the extensibility of the code, if we were to allow the Crater class to spawn other creatures in future development, and we know that all spawners would implement the CreatureSpawnable interface, no modifications need to be made to Crater to allow spawning of different creatures (Open-Closed Principle). Adding the CreatureSpawnable interface also allows us to segregate the responsibility of spawning creatures and spawning fruits (FruitSpawnable in REQ 2), with different types, it is a more logical approach to have different interfaces that handle spawning of different types (Interface Segragation Principle).

Adding a new CreatureAttackBehaviour allows us to segregate the responsibility of checking if an actor of certain status is in the surroundings of a creature and adding a new AttackAction if

there is a target in vicinity, rather than simply adding the new logic to the playTurn() method of the creature. Using this implementation, each class is responsible for their own respective purpose only, different responsibilities are distributed to different classes (Single Responsibility Principle).

HuntsmanSpider class, Player class and CreatureAttackBehaviour class all depend on enum Status class, which acts as an indicator to allow certain actions. Using the dependency on the enum Status class, we avoided using downcasting to check for the types of the concrete classes to determine if the actions are allowed on objects of these concrete classes, resulting in easier functionality extensions without modifying existing code with additional if-else checks (Open-Closed Principle).

REQ 4

A. Implementation of special scrap MetalPipeItem
   Create new class MetalPipeItem that extends abstract Item class from engine. Override existing allowableActions() method to allow the item itself to add an attack action to the allowable actions of the actor that has the item.

Pros	Cons
Increases code reusability as it uses existing abstract classes to reduce repeated code and obtain similar attributes and methods by inheritance.	When many special items allow the same actions, this would introduce many blocks of repeated code for the allowableActions() method.
MetalPipeItem class is responsible for adding an attack action using itself to the options, reducing dependencies of other classes on MetalPipeItem class by segregating this responsibility.	
More flexible, expanding new special items can also be done by extending abstract Item class and adding their own allowed actions, which also adds versatility of special item effects.	

B. Player to attack a nearby HuntsmanSpider with default bare hands
   Override the existing getIntrinsicWeapon() method of the inherited Actor class to return the intrinsic weapon of the player with new defined attribute values other than using the default given.

Pros	Cons
Reuses available codebase, using IntrinsicWeapon class given allows integration of the engine classes with the game, increasing cohesion of the system.	Code needs be modified in multiple classes to extend the system to accommodate more than one intrinsic weapon per actor
Versatile implementation of various types of intrinsic weapons for different actors.	

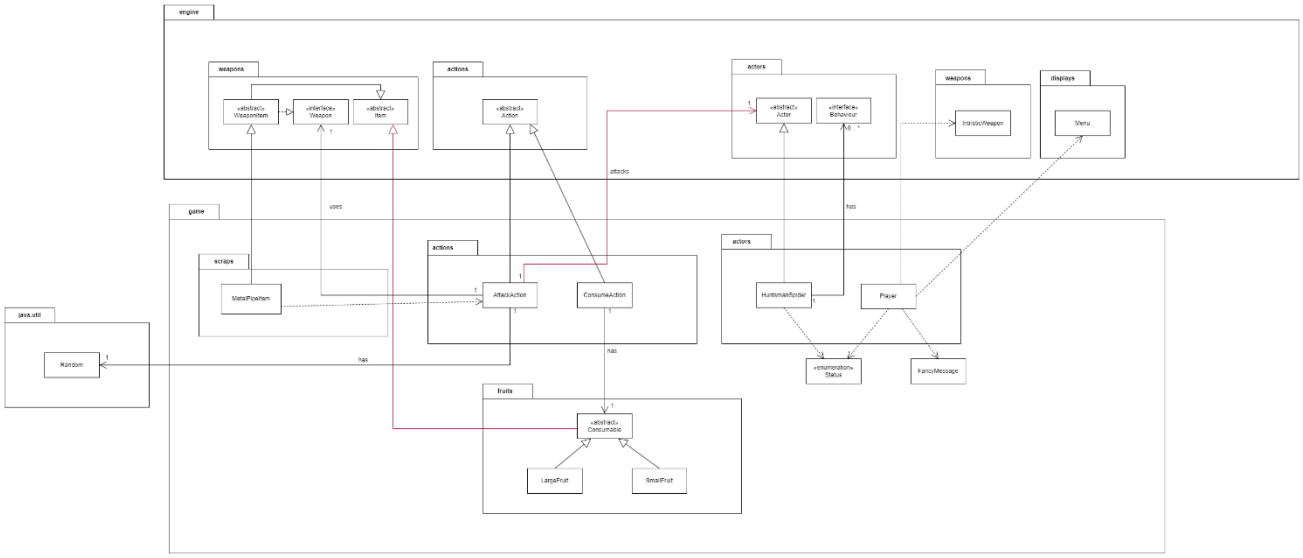
C. Implementation of items that can heal actor when consumed

Design 1: Create interface Consumable, which can be implemented by the abstract Fruit class, which is the base for concrete fruit items.

Pros	Cons
Increased flexibility in defining consumables, any class can implement the Consumable interface to indicate that it is consumable, regardless of its inheritance hierarchy.	Increased repeated code as allowableActions() method for Consumable items implement same or similar logic, which limits code reusability.

Design 2: Create abstract class Consumable that extends abstract Item class from engine, which can be inherited by any class that represents consumable items.

Pros	Cons
Easier to extend common attributes and methods by adding to the Consumable abstract class, without need of modifications to its subclasses.	Less flexible and reduced reusability to other classes rather than Item, i.e. if other entities which are not items can be consumable, we need to create new logic to implement this feature.
Inheritance of an abstract class reduces code duplications as common behaviours and attributes are inherited by its subclasses.	



The diagram above shows the final design of requirement 4, which utilizes designs of part A, B with Design 2 of part C stated above to implement a special scrap Metal Pipe, that can be used to attack hostile creatures, allow attacking without special scraps and implement special items that can be consumed to obtain certain healing effects.

The new MetalPipeItem class extends abstract WeaponItem class, Consumable class extends abstract class Item and ConsumeAction class extends the abstract Action class. All these classes inherit their methods and attributes from existing abstract classes provided in the engine. We abstract these identities to avoid repetitions in code (DRY), as these classes share many same attributes and methods with the respective abstract classes they extend from.

In MetalPipeItem class, the allowableActions() method is overridden to add a new attack action when meeting a hostile creature, giving MetalPipeItem class the responsibility of adding an attack action using itself to the options, reducing dependencies of other classes on this class by segregating this responsibility (Single Responsibility Principle).

Reusing the given IntrinsicWeapon class of the engine to add a default weapon of using bare hands to the Player class promotes code reuse and reduces duplicating logic in the system (DRY). Overriding the existing getIntrinsicWeapon() method of the inherited Actor class to return the intrinsic weapon of the player with new defined attribute values was designed in care to function just as how the function was defined in the base class Actor, allowing the subclass to demonstrate changes to the intrinsic weapon involved without changing the predefined behaviour of the method in the base class (Liskov Substitution Principle).

Creating the abstract class Consumable which can be inherited by any class that represents consumable items provides a clear contract of which items can be consumed, avoiding the use of downcasting in code to check for items which are consumable, extending this abstract would indicate that the item is consumable (Open-Closed Principle). Inheritance of an abstract class reduces code duplications as common behaviours and attributes are inherited by its subclasses, compared to using interfaces as consumable items mostly implement the

same behaviours (DRY). Using abstract Consumable class also allows us to achieve less dependencies of the ConsumeAction class on concrete consumable item classes, with al dependencies depending on the abstract class (Dependency Inversion Principle).
Creating a new ConsumeAction class allows the responsibility of performing a consume on the item to be transferred to the ConsumeAction class, rather than allowing the Consumable class manage the execution of consuming, which promotes segregation of responsibilities to different classes (Single Responsibility Principle).